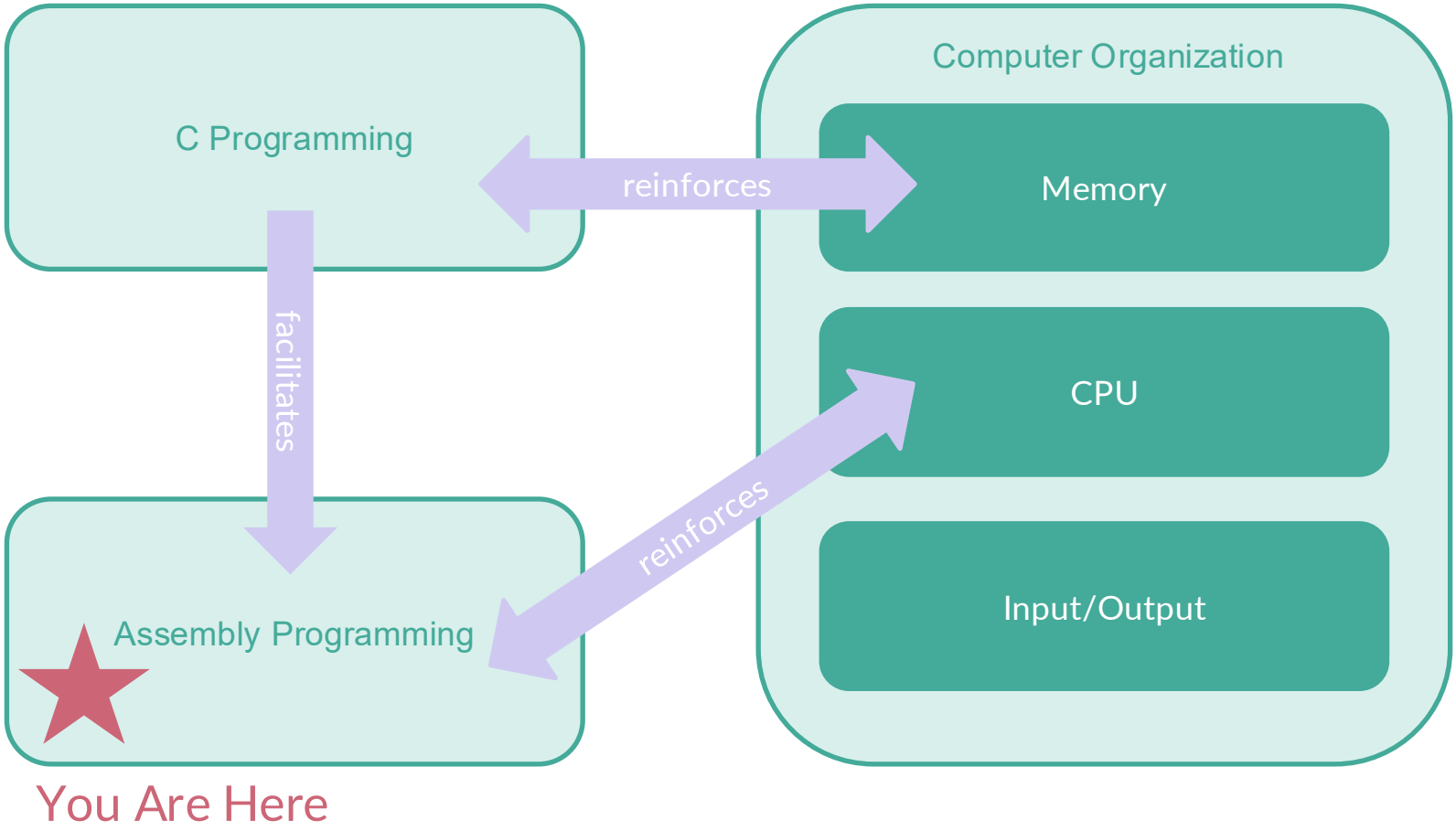
CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

Introduction to ISAs and Assembly

Components of the Course



Assembly Bridges Software and Hardware

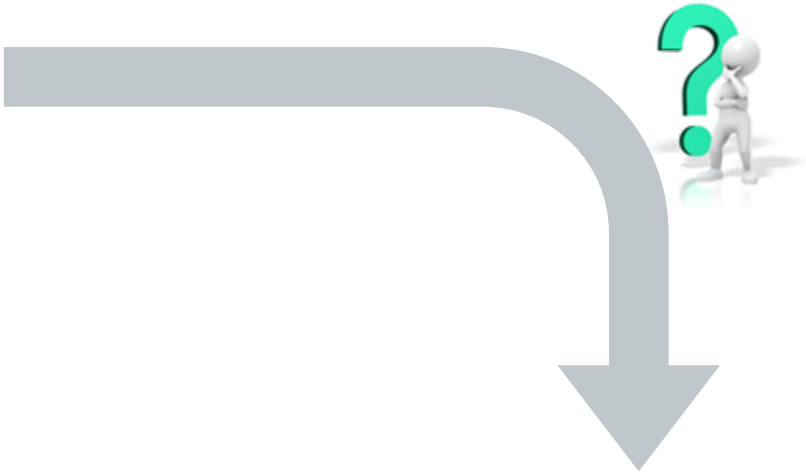
```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char* argv[])
{
  int iter;
  double x, y, z;

  printf("Iterations: ");
  scanf("%d",&iter);

  /* initialize random numbers */
  srand(123456789);

  /* cnt = number of points in the
     1st quadrant of unit circle */
  int cnt = 0;
  int i = 0;
  while (i++ < iter){
    x = (double)rand() / RAND_MAX;
    y = (double)rand() / RAND_MAX;
    z = x*x + y*y;
    if (z <= 1) cnt++;
  }
  double pi=(double)cnt / iter * 4;
  printf("pi is %g\n", pi);
  return 0;
}
```





Why Does This Matter?

Understanding assembly helps you:

- debug memory errors and program crashes
- optimize code for maximum performance
- implement system software
- analyze and patch security vulnerabilities

REVIEW SUMMARY

COMPUTER SCIENCE

There's plenty of room at the Top: What will drive computer performance after Moore's law?

Charles E. Leiserson, Neil C. Thompson*, Joel S. Emer, Bradley C. Kuszmaul, Butler W. Lampson, Daniel Sanchez, Tao B. Schardl

BACKGROUND: Improvements in computing power can claim a large share of the credit for many of the things that we take for granted in our modern lives: cellphones that are more powerful than room-sized computers from 25 years ago, internet access for nearly half the world, and drug discoveries enabled by powerful supercomputers. Society has come to rely on computers whose performance increases exponentially over time.

Much of the improvement in computer performance comes from decades of miniaturization of computer components, a trend that was foreseen by the Nobel Prize–winning physicist Richard Feynman in his 1959 address, "There's Plenty of Room at the Bottom," to the American Physical Society. In 1975, Intel founder Gordon Moore predicted the regularity of this miniaturization trend, now called Moore's law, which, until recently, doubled the number of transistors on computer chips every 2 years.

Unfortunately, semiconductor miniaturization is running out of steam as a viable way to grow computer performance—there isn't much more room at the "Bottom." If growth in computing power stalls, practically all industries will face challenges to their productivity. Nevertheless, opportunities for growth in computing performance will still be available, especially at the "Top" of the computing-technology stack: software, algorithms, and hardware architecture.

ADVANCES: Software can be made more efficient by performance engineering: restructuring software to make it run faster. Performance engineering can remove inefficiencies in programs, known as software bloat, arising from traditional software-development strategies that aim to minimize an application's development time rather than the time it takes to run. Performance engineering can also tailor software to the hardware on which it runs, for example, to take advantage of parallel processors and vector units.

Algorithms offer more-efficient ways to solve problems. Indeed, since the late 1970s, the time to solve the maximum-flow problem improved nearly as much from algorithmic advances as from hardware speedups. But progress on a given algorithmic problem occurs unevenly and sporadically and must ultimately face diminishing returns. As such, we see the biggest benefits coming from algorithms for new problem domains (e.g., machine learning) and from developing new theoretical machine models that better reflect emerging hardware.

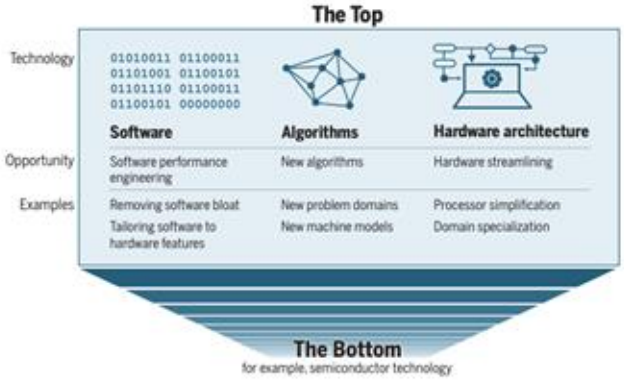
ON OUR WEBSITE

Read the full article at https://dx.doi.org/10.1126/science.aam9744

Hardware architectures can be streamlined—for instance, through processor simplification, where a complex processing core is replaced with a simpler core that requires fewer transistors. The freed-up transistor budget can then be redeployed in other ways—for example, by increasing the number of processor cores running in parallel, which can lead to large efficiency gains for problems that can exploit parallelism. Another form of streamlining is domain specialization, where hardware is customized for a particular application domain. This type of specialization jettisons processor functionality that is not needed for the domain. It can also allow more customization to the specific characteristics of the domain, for instance, by decreasing floating-point precision for machine-learning applications.

In the post-Moore era, performance improvements from software, algorithms, and hardware architecture will increasingly require concurrent changes across other levels of the stack. These changes will be easier to implement, from engineering-management and economic points of view, if they occur within big system components: reusable software with typically more than a million lines of code or hardware of comparable complexity. When a single organization or company controls a big component, modularity can be more easily reengineered to obtain performance gains. Moreover, costs and benefits can be pooled so that important but costly changes in one part of the big component can be justified by benefits elsewhere in the same component.

OUTLOOK: As miniaturization wanes, the silicon-fabrication improvements at the Bottom will no longer provide the predictable, broad-based gains in computer performance that society has enjoyed for more than 50 years. Software performance engineering, development of algorithms, and hardware streamlining at the Top can continue to make computer applications faster in the post-Moore era. Unlike the historical gains at the Bottom, however, gains at the Top will be opportunistic, uneven, and sporadic. Moreover, they will be subject to diminishing returns as specific computations become better explored. ■



Performance gains after Moore's law ends. In the post-Moore era, improvements in computing power will increasingly come from technologies at the "Top" of the computing stack, not from those at the "Bottom", reversing the historical trend.

The list of author affiliations is available in the full article online.
*Corresponding author. Email: neil_t@mit.edu
Cite this article as C. E. Leiserson et al., Science 368, eaam9744 (2020). DOI: 10.1126/science.aam9744

link

REVIEW SUMMARY

COMPUTER SCIENCE

There's plenty of room at the Top: What will drive computer performance after Moore's law?

Charles E. Leiserson, Neil C. Thompson*, Joel S. Emer, Bradley C. Kuszmaul, Butler W. Lampson, Daniel Sanchez, Tao B. Schardl

BACKGROUND: Improvements in computing | in computing power stalls, practically all in-

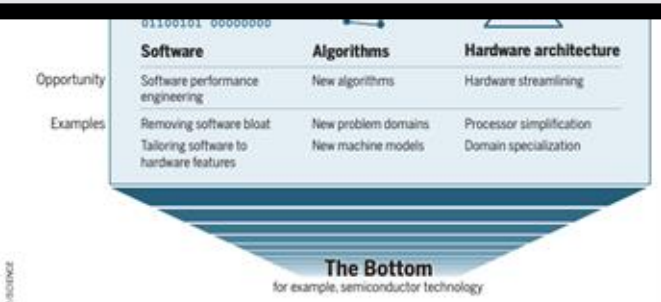
and sporadically and must ultimately face diminishing returns. As such, we see the biggest benefits coming from algorithms for new problem domains (e.g., machine learning) and from developing new theoretical machine models that better reflect emerging hardware. Hardware architectures can be streamlined—for instance, through processor simplification, where a complex processing core is replaced with a simpler core that requires fewer transistors. The freed-up transistor budget can

ON OUR WEBSITE

Read the full article at https://dx.doi.org/10.1126/science.aam9744

Table 1. Speedups from performance engineering a program that multiplies two 4096-by-4096 matrices. Each version represents a successive refinement of the original Python code. "Running time" is the running time of the version. "GFLOPS" is the billions of 64-bit floating-point operations per second that the version executes. "Absolute speedup" is time relative to Python, and "relative speedup," which we show with an additional digit of precision, is time relative to the preceding line. "Fraction of peak" is GFLOPS relative to the computer's peak 835 GFLOPS. See Methods for more details.

Version	Implementation	Running time (s)	GFLOPS	Absolute speedup	Relative speedup	Fraction of peak (%)
1	Python	25,552.48	0.005	1	–	0.00
2	Java	2,372.68	0.058	11	10.8	0.01
3	C	542.67	0.253	47	4.4	0.03
4	Parallel loops	69.80	1.969	366	7.8	0.24
5	Parallel divide and conquer	3.80	36.180	6,727	18.4	4.33
6	plus vectorization	1.10	124.914	23,224	3.5	14.96
7	plus AVX intrinsics	0.41	337.812	62,806	2.7	40.45



Performance gains after Moore's law ends. In the post-Moore era, improvements in computing power will increasingly come from technologies at the "Top" of the computing stack, not from those at the "Bottom", reversing the historical trend.

fabrication improvements at the Bottom will no longer provide the predictable, broad-based gains in computer performance that society has enjoyed for more than 50 years. Software performance engineering, development of algorithms, and hardware streamlining at the Top can continue to make computer applications faster in the post-Moore era. Unlike the historical gains at the Bottom, however, gains at the Top will be opportunistic, uneven, and sporadic. Moreover, they will be subject to diminishing returns as specific computations become better explored. ■

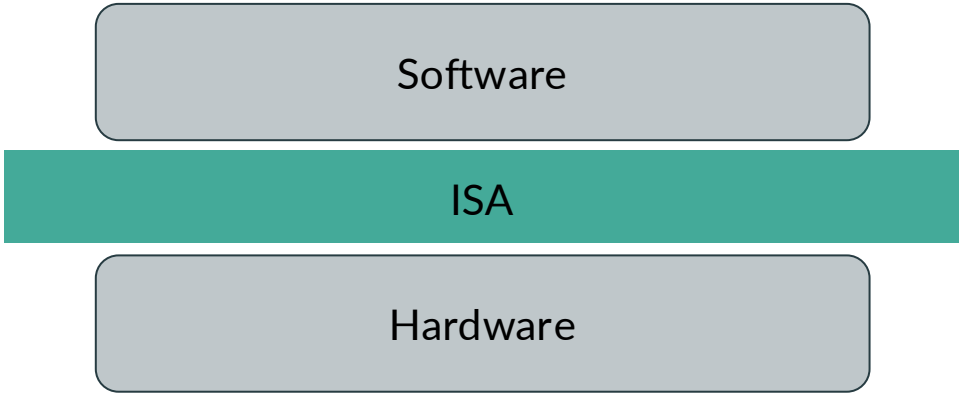
The list of author affiliations is available in the full article online.
*Corresponding author. Email: neil_t@mit.edu
Cite this article as C. E. Leiserson et al., Science 368, eaam9744 (2020). DOI: 10.1126/science.aam9744

link

Instruction Set Architecture

What is an ISA?

The **I**nstruction **S**et **A**rchitecture is the interface between software and hardware



It is defined by

- instructions: the language used to communicate
- architectural state: the registers and memory instructions can operate on

The ISA Specifies What, Not How

The **architecture** specifies what the computer does

- What instructions can it execute?
- How are instruction operands specified?
- How much memory does it have?
- How is the memory accessed?

The ISA Specifies What, Not How

The **architecture** specifies what the computer does

- What instructions can it execute?
- How are instruction operands specified?
- How much memory does it have?
- How is the memory accessed?

The **microarchitecture** specifies how the computer does it

- How is an instruction implemented?
- How long does each instruction take?
- How long does it take to access memory?

Real World ISAs

- x86
  - Intel owns and maintains the specification
  - Intel and AMD design and sell processors that implement x86
- ARM
  - used in embedded devices
  - Apple iPhones
- PowerPC
  - Sony PS3
  - old Apple Macs
- RISC-V
- SPARC
- HP-PA
- Alpha
- Motorola 68000
- DEC VAX
- ...

A Brief History of Computing

Before the 1960s every computer had its own bespoke ISA



Every computer needed custom programs



A Brief History of Computing

In 1965 the IBM System/360 separated the architecture from the microarchitecture

Many different implementations of the same interface means that the same code can be reused



A Brief History of Computing

ISAs are designed to support high-level programming constructs



Instructions are very expressive, but that makes processors slower and more complicated



A Brief History of Computing

ISAs are designed to support high-level programming constructs

Instructions are very expressive, but that makes processors slower and more complicated

*this is retroactively called **CISC**: **C**omplex **I**nstruction **S**et **C**omputer



A Brief History of Computing

In the 1980s high-level programming languages flourish and compilers mature



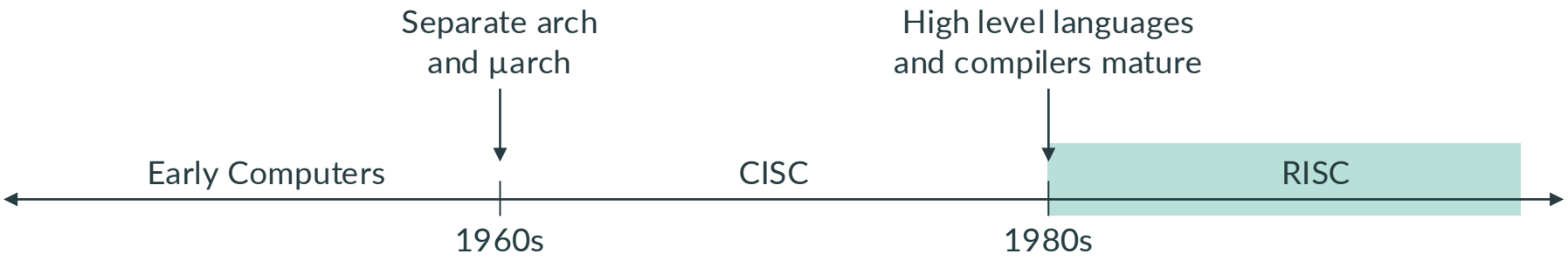
Programmers write more code in high-level languages, so ISAs can be optimized for execution rather than coding



A Brief History of Computing

New Idea: Design ISAs with simple instructions to support better processor implementations

Assembly programs need more instructions, but each instruction executes fast enough to offset the longer program

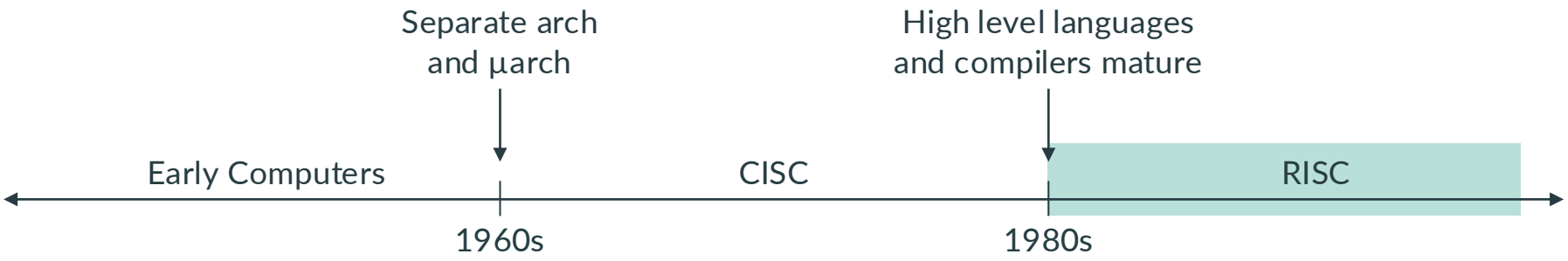


A Brief History of Computing

New Idea: Design ISAs with simple instructions to support better processor implementations

Assembly programs need more instructions, but each instruction executes fast enough to offset the longer program

*this is called **RISC**: **R**educed **I**nstruction **S**et **C**omputer



RISC vs. CISC

	RISC	CISC
Instruction Complexity	simple, fixed-length instructions	complex, variable-length instructions
Number of instructions	small set of operations	large set of operations
Execution time	all instructions execute in a single clock cycle	different instructions have different execution time
Memory access	only load/store instructions access memory	many instructions access memory

RISC ISA Design Principles

Simplicity favors regularity: If instructions are simple and uniform they are easier to execute in hardware.

RISC ISA Design Principles

Simplicity favors regularity: If instructions are simple and uniform they are easier to execute in hardware.

Make the common case fast: Optimize the instruction set to do common operations as fast as possible.

RISC ISA Design Principles

Simplicity favors regularity: If instructions are simple and uniform they are easier to execute in hardware.

Make the common case fast: Optimize the instruction set to do common operations as fast as possible.

Smaller is faster: A smaller, more limited set of options can be implemented as a smaller, faster circuit.

RISC ISA Design Principles

Simplicity favors regularity: If instructions are simple and uniform they are easier to execute in hardware.

Make the common case fast: Optimize the instruction set to do common operations as fast as possible.

Smaller is faster: A smaller, more limited set of options can be implemented as a smaller, faster circuit.

Good design demands good compromises: Sometimes these principles make operations harder or slower, and that's fine.

Assembly Programming

What is Assembly Language?

Assembly language: ISA instructions in human-readable format

Machine language: ISA instructions in computer-readable format (i.e., encoded in binary)

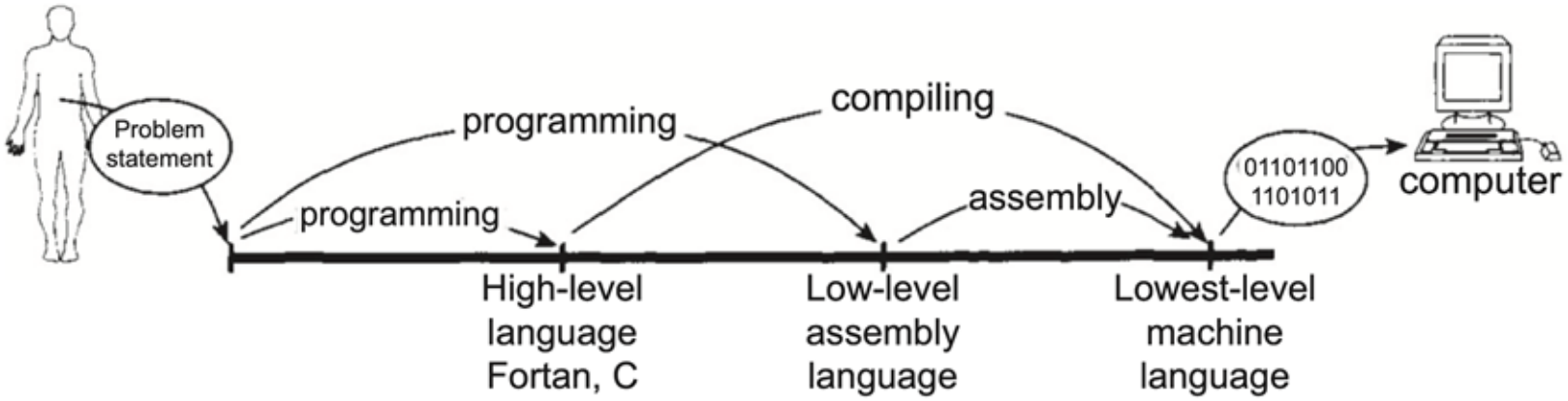


Diagram source: Plonus, "The digital computer"

Example Algorithm: Euclid's GCD

The greatest common divisor of two numbers does not change if the smaller number is subtracted from the larger number.

Example: Find the GCD of 49 and 21

Euclid's GCD Algorithm

1. Call the two numbers a and b
2. If a and b are equal, stop: a is the greatest common divisor
3. Subtract the smaller from the larger
4. Repeat steps 2–4

```
int gcd(int a, int b) {
  while (a != b) {
    if (a > b) a = a-b;
    else       b = b-a;
  }
  return a;
}
```

A More Explicit GCD Algorithm

1. Call the two numbers a and b
2. If a equals b, go to step 8
3. If a is less than b, go to step 6
4. Subtract b from a
5. Go to step 2
6. Subtract a from b
7. Go to step 2
8. Declare a the greatest common divisor
9. Go back to previous work

Euclid's Algorithm in MIPS Assembly

Every instruction is a single primitive operation.

```
gcd:
        beq  $a0, $a1, exit # if a = b, go to exit
        slt  $v0, $a1, $a0  # is b > a?
        bne  $v0, $0, suba  # if yes, goto suba

        subu $a0, $a0, $a1  # subtract b from a
        b    gcd            # and repeat

suba:
        subu $a1, $a1, $a0  # subtract a from b
        b    gcd            # and repeat

exit:
        move $v0, $a0       # return a
        jr   $ra            # go back to caller
```

Euclid's Algorithm in MIPS Assembly

Opcodes specify the operation the CPU should perform.

```
gcd:
        beq  $a0, $a1, exit # if a = b, go to exit
        slt  $v0, $a1, $a0  # is b > a?
        bne  $v0, $0, suba  # if yes, goto suba

        subu $a0, $a0, $a1  # subtract b from a
        b    gcd            # and repeat

suba:
        subu $a1, $a1, $a0  # subtract a from b
        b    gcd            # and repeat

exit:
        move $v0, $a0       # return a
        jr   $ra            # go back to caller
```

Euclid's Algorithm in MIPS Assembly

Operands specify the data or location the operation acts on.

```
gcd:
        beq  $a0, $a1, exit # if a = b, go to exit
        slt  $v0, $a1, $a0  # is b > a?
        bne  $v0, $0, suba  # if yes, goto suba

        subu $a0, $a0, $a1  # subtract b from a
        b    gcd            # and repeat

suba:
        subu $a1, $a1, $a0  # subtract a from b
        b    gcd            # and repeat

exit:
        move $v0, $a0       # return a
        jr   $ra            # go back to caller
```

Euclid's Algorithm in MIPS Assembly

Labels name a specific location in the program (used for reference).

```
gcd:
        beq  $a0, $a1, exit # if a = b, go to exit
        slt  $v0, $a1, $a0  # is b > a?
        bne  $v0, $0, suba  # if yes, goto suba

        subu $a0, $a0, $a1  # subtract b from a
        b    gcd            # and repeat

suba:
        subu $a1, $a1, $a0  # subtract a from b
        b    gcd            # and repeat

exit:
        move $v0, $a0       # return a
        jr   $ra            # go back to caller
```

Euclid's Algorithm in MIPS Assembly

Comments explain what instructions mean in human terms.

```
gcd:
        beq  $a0, $a1, exit # if a = b, go to exit
        slt  $v0, $a1, $a0  # is b > a?
        bne  $v0, $0, suba  # if yes, goto suba

        subu $a0, $a0, $a1  # subtract b from a
        b    gcd            # and repeat

suba:
        subu $a1, $a1, $a0  # subtract a from b
        b    gcd            # and repeat

exit:
        move $v0, $a0       # return a
        jr   $ra            # go back to caller
```

C → Assembly → Machine Code

```
int gcd(int a, int b) {
  while (a != b) {
    if (a > b) a = a-b;
    else       b = b-a;
  }
  return a;
}
```

Compiler

```
gcd:
        beq  $a0, $a1, exit
        sgt  $v0, $a1, $a0
        bne  $v0, $0, suba
        subu $a0, $a0, $a1
        b    gcd
suba:
        subu $a1, $a1, $a0
        b    gcd
exit:
        move $v0, $a0
        jr   $ra
```

Assembler

```
beq  $4, $5, 28
slt  $2, $5, $4
bne  $2, $0, 12
subu $4, $5, $4
bgez $0, -16
subu $4, $4, $5
bgez $0, -24
addu $2, $0, $4
jr   $31
```

```
00010000100001010000000000000111
00000000101001000001000000101010
00010100010000000000000000000011
00000000101001000010100000100011
00000100000000001111111111111100
00000000100001010010000000100011
00000100000000001111111111111010
00000000000001000001000000100001
00000011111100000000000000001000
```

Exercise: Assembly

MIPS Assembly

What is MIPS?

Microprocessor without **I**nterlocked **P**ipeline **S**tages

MIPS ISA developed by John Hennessey and David Patterson (and others)

We will use 32-bit MIPS, meaning word size is 32 bits

- RISC architecture
- Load/store architecture
- Three-operand format
- Explicit control flow

2017 Turing Award

Hennessy and Patterson were recognized with the Turing Award for “pioneering a systematic, quantitative approach to the design and evaluation of computer architectures with enduring impact on the microprocessor industry.”



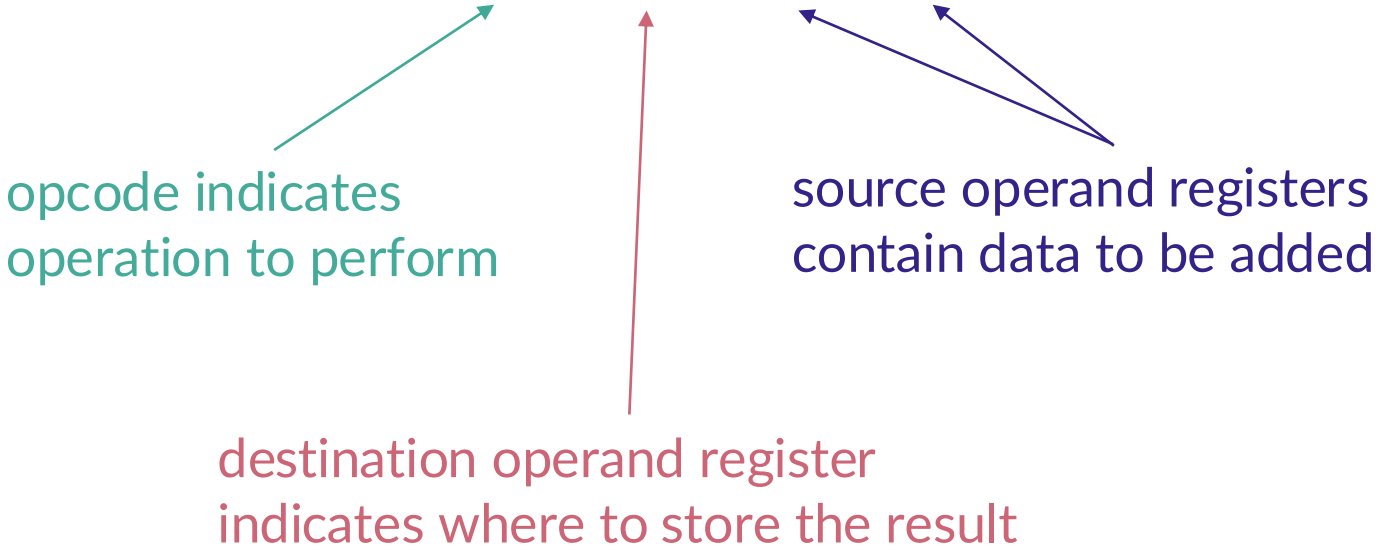
Addition in MIPS

C code:

```
a = b + c;
```

MIPS assembly code:

```
add $a, $b, $c
```



Subtraction in MIPS

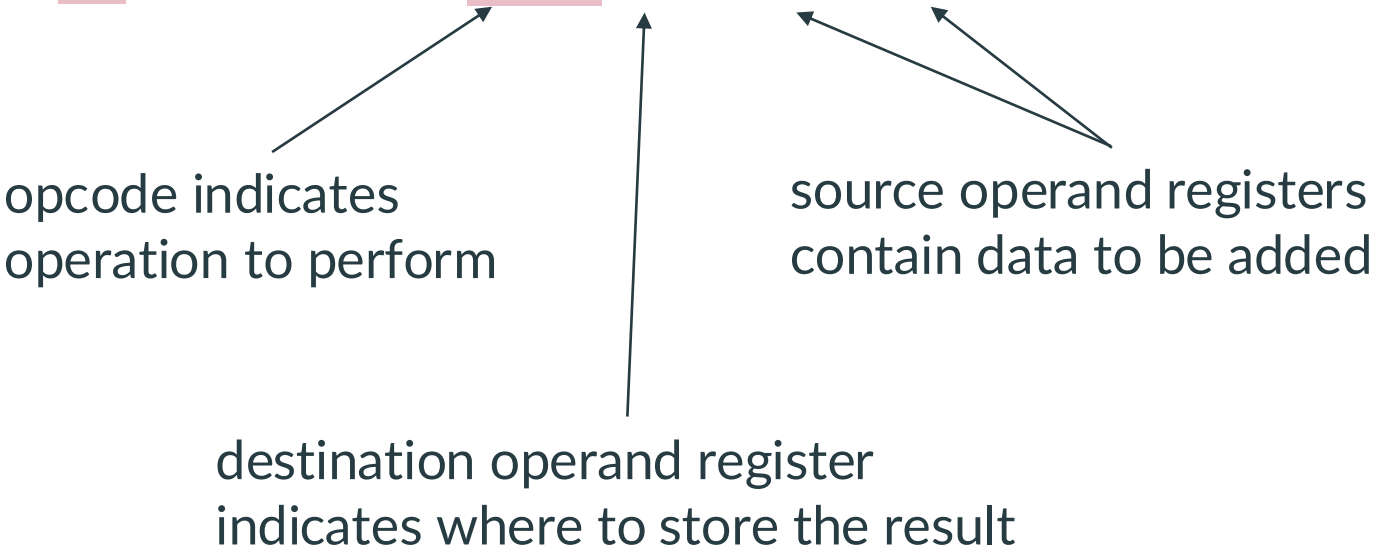
From addition to subtraction only the opcode has changed.

C code:

```
a = b - c;
```

MIPS assembly code:

```
sub $a, $b, $c
```



Subtraction in MIPS

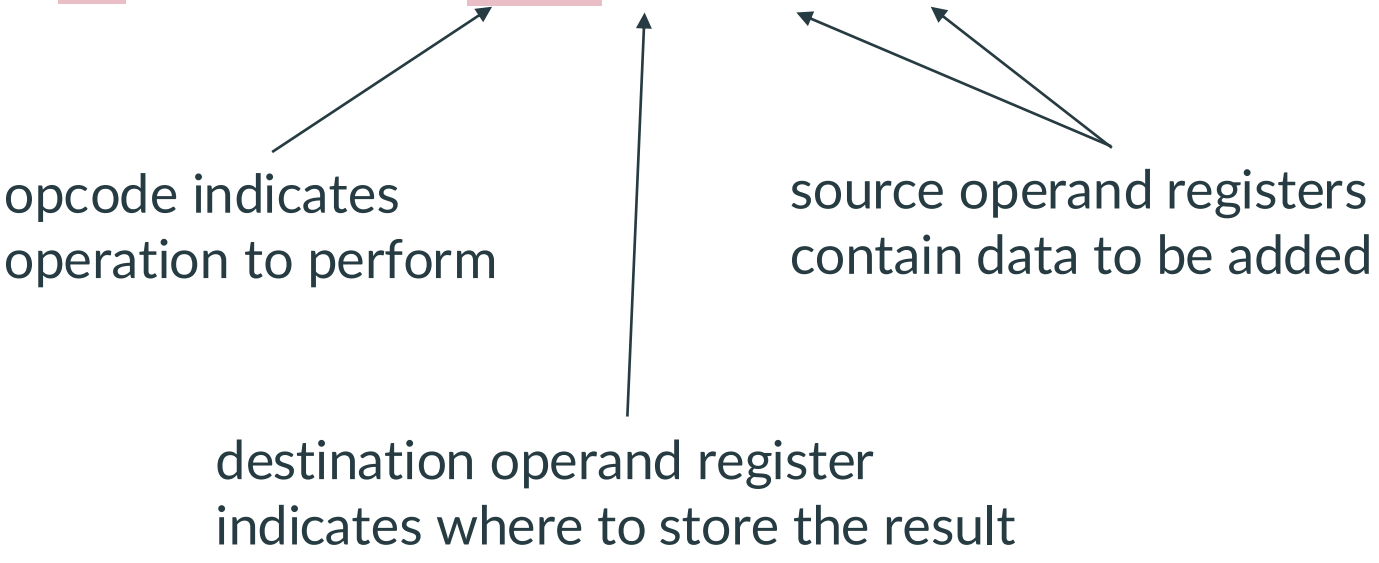
From addition to subtraction only the opcode has changed.

C code:

```
a = b - c;
```

MIPS assembly code:

```
sub $a, $b, $c
```



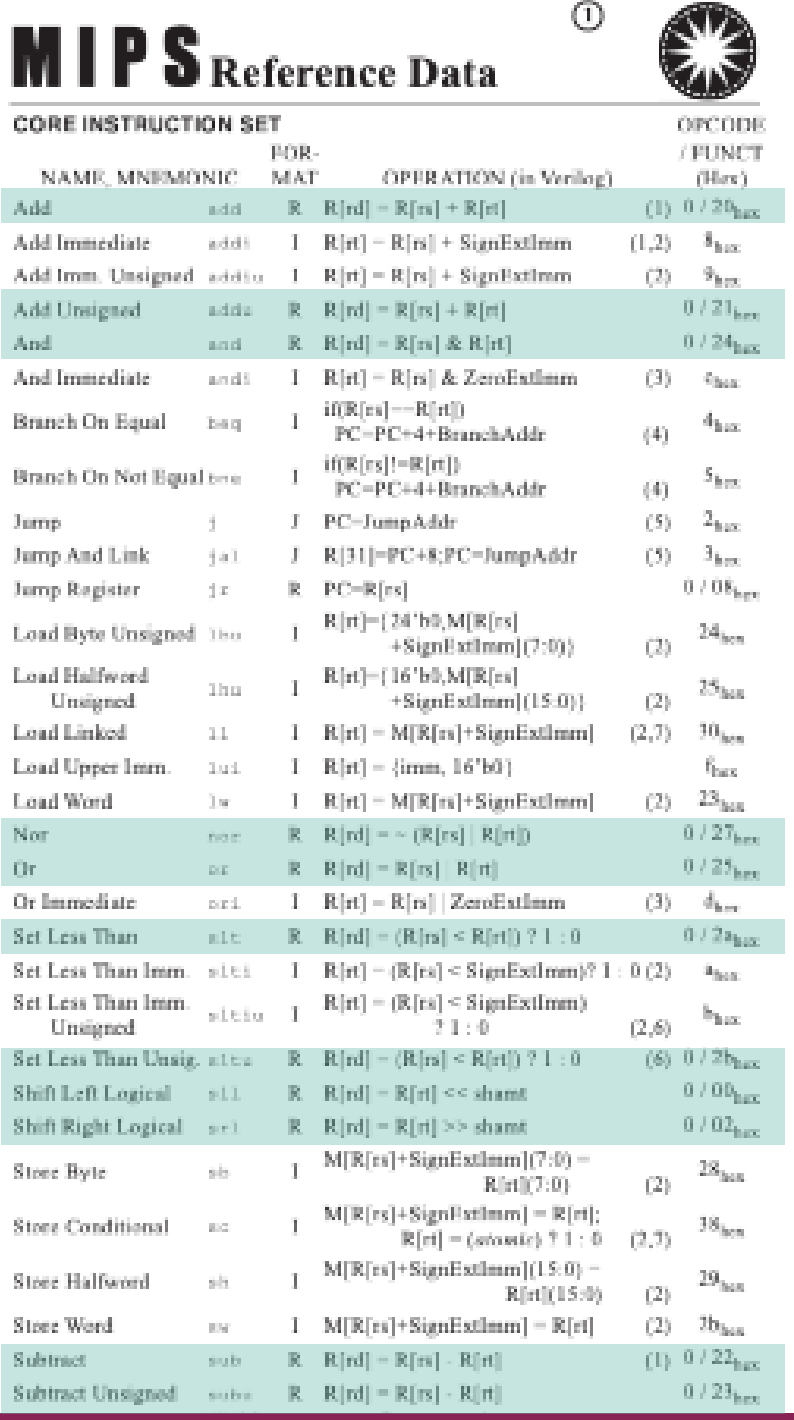
RISC Design Principle #1: Simplicity favors regularity

- Consistent instruction format
- Consistent operand order

Computational Instructions

operation $rd, $rs, $rt

“perform operation on the contents of registers $rs and $rt and put the result in $rd”



MIPS Reference Data ①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	0 / 20_{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		0 / 21_{hex}
And	and	R	R[rd] = R[rs] & R[rt]		0 / 24_{hex}
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		0 / 08_{hex}
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs]+SignExtImm](7:0)}	(2)	24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs]+SignExtImm](15:0)}	(2)	25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7)	30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2)	23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])		0 / 27_{hex}
Or	or	R	R[rd] = R[rs] | R[rt]		0 / 25_{hex}
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3)	d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		0 / $2a_{hex}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2)	a_{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6)	b_{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	0 / $2b_{hex}$
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt		0 / 00_{hex}
Shift Right Logical	srl	R	R[rd] = R[rt] >> shamt		0 / 02_{hex}
Store Byte	sb	I	M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2)	28_{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt]; R[rt] = (atomic) ? 1 : 0	(2,7)	38_{hex}
Store Halfword	sh	I	M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2)	29_{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt]	(2)	$2b_{hex}$
Subtract	sub	R	R[rd] = R[rs] - R[rt]	(1)	0 / 22_{hex}
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt]		0 / 23_{hex}

Complex Code ➔ Multiple Instructions

More complex code is handled by multiple instructions.

C code:

```
a = b + c - d;
```

MIPS assembly code:

```
add $t, $b, $c  # t = b + c
sub $a, $t, $d  # a = t - d
```

Complex Code ➔ Multiple Instructions

More complex code is handled by multiple instructions.

C code:

```
a = b + c - d;
```

MIPS assembly code:

```
add $t, $b, $c  # t = b + c
sub $a, $t, $d  # a = t - d
```

RISC Design Principle #2: Make the common case fast

- MIPS includes only simple, commonly used instructions
- Support more complex and less common operations via multiple instructions

Additional MIPS Resources

MIPS reference card

Short Tutorial

Extensive Tutorial

Simulating MIPS Code

We will use mipsy: https://cgi.cse.unsw.edu.au/~cs1521/mipsy/

